

Juego de fichas

Beto y Carlos están jugando un juego en una cuadrícula de $2N \times 2M$ celdas cuadradas de una unidad. Las filas están numeradas de 0 a $2N - 1$ de arriba hacia abajo, y las columnas están numeradas de 0 a $2M - 1$ de izquierda a derecha. Para $0 \leq i < 2N$ y $0 \leq j < 2M$, denotamos la celda en la fila i y la columna j como (i, j) .

Beto le da a Carlos $N \cdot M$ **bloques**, uno por uno. Cada bloque es un cuadrado de 2×2 que consta de cuatro **fichas** de 1×1 . Beto ha coloreado cada ficha del bloque de negro o blanco, asegurándose de que **al menos una ficha sea blanca**.

Carlos debe colocar cada bloque en la cuadrícula **inmediatamente después de recibirlo**, sin saber qué bloques recibirá más adelante. Los bloques no se pueden girar. Cada bloque debe colocarse completamente dentro de la cuadrícula, cubriendo exactamente cuatro celdas. Además, la ficha superior izquierda de cada bloque debe cubrir una celda cuyas coordenadas de fila y columna sean ambas pares. Cada celda de la cuadrícula puede ser cubierta por un máximo de un bloque.

Beto gana el juego si, después de colocar cualquier bloque, existe un cuadrado de celdas 2×2 cubierto por cuatro fichas negras. Formalmente, si las celdas (a, b) , $(a + 1, b)$, $(a, b + 1)$, $(a + 1, b + 1)$ están todas cubiertas por fichas negras para algún $0 \leq a < 2N - 1$ y $0 \leq b < 2M - 1$, entonces Beto gana. Los índices a y b no necesitan ser pares.

Carlos gana si coloca todos los bloques $N \cdot M$ sin que Beto gane en ningún momento. Ten en cuenta que colocar los bloques $N \cdot M$ cubrirá completamente la cuadrícula.

Tu tarea es implementar una estrategia para que Carlos gane el juego. Se puede demostrar que, bajo las restricciones dadas, Carlos siempre puede colocar los bloques de manera que se garantice la victoria, sin importar los colores de los bloques que reciba más adelante.

Detalles de implementación

Debes implementar dos procedimientos:

```
void init(int N, int M);
```

- N : la mitad del número de filas de la cuadrícula.
- M : la mitad del número de columnas de la cuadrícula.

- El procedimiento se llama exactamente una vez por cada caso de prueba, al inicio de la ejecución de tu programa.

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR)
```

- TL, TR, BL, BR : los colores de las casillas superior izquierda, superior derecha, inferior izquierda e inferior derecha del bloque actual, respectivamente, como se muestra en la figura a continuación. Cada valor es 0 (blanco) o 1 (negro).
- Este procedimiento se invoca exactamente $N \cdot M$ veces por cada caso de prueba, después de la llamada inicial a `init`.



Este procedimiento debe devolver un par de números enteros (i, j) , donde i es la coordenada de fila y j es la coordenada de columna de la celda donde debe colocarse el mosaico superior izquierdo de este bloque. Tanto i como j deben ser pares, y la región 2×2 cubierta por el bloque no debe superponerse a ningún bloque colocado previamente.

Si `receive_block` devuelve en algún momento un par que no cumpla con estos requisitos, o si después de colocar el bloque un cuadrado de celdas 2×2 queda completamente cubierto por fichas negras, el evaluador terminará inmediatamente tu programa y el veredicto para el caso de prueba será `Output isn't correct`.

El comportamiento del evaluador **no es adaptativo**. Esto significa que la secuencia de bloques que Beto le da a Carlos está fija antes de que se llame `ainit`.

Restricciones

Para cada bloque, sea S el número de fichas negras entre sus cuatro fichas. Es decir, $S = TL + TR + BL + BR$.

- $1 \leq N, M \leq 100$
- $0 \leq S \leq 3$ para cada bloque.

Subtareas

Subtarea	Puntuación	Restricciones adicionales
1	6	$S = 1$ para cada bloque y $N = 2$.
2	16	$S = 3$ para cada bloque. $N = M$, N es par, y cada una de las cuatro coloraciones posibles de los bloques aparece exactamente $\frac{N^2}{4}$ veces.
3	10	$S = 1$ para cada bloque.
4	29	$S \leq 2$ para cada bloque.
5	39	No hay restricciones adicionales.

Ejemplo

Consideremos un juego con $N = 1$ y $M = 2$, por lo que la cuadrícula tiene 2 filas y 4 columnas. El evaluador primero llama a:

```
init(1, 2)
```

Inicialmente, todas las celdas están vacías. La cuadrícula se ve así:

	0	1	2	3
0				
1				

Hay $N \cdot M = 2$ bloques para colocar. Supongamos que Beto entrega un bloque con tres fichas negras y una ficha blanca en la esquina superior derecha. El evaluador llama a:

```
receive_block(1, 0, 1, 1)
```

Carlos decide colocar este bloque en el lado izquierdo de la cuadrícula devolviendo $(0, 0)$.

La cuadrícula ahora se ve así:

	0	1	2	3
0				
1				

Luego, Beto entrega otro bloque con tres fichas negras y una ficha blanca en la esquina superior izquierda:

```
receive_block(0, 1, 1, 1)
```

La única celda restante con fila par y columna par que puede servir como esquina superior izquierda de un bloque 2×2 es $(0, 2)$, por lo que Carlos devuelve $(0, 2)$. La cuadrícula queda así:

	0	1	2	3
0				
1				

Ningún cuadrado de 2×2 está completamente cubierto por fichas negras, por lo que Carlos ha colocado con éxito todos los bloques sin que Beto haya ganado en ningún momento. Carlos gana el juego.

Evaluador de ejemplos

Formato de entrada:

```
N M
TL[0] TR[0] BL[0] BR[0]
TL[1] TR[1] BL[1] BR[1]
...
TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

Formato de salida:

```
R[0] C[0]  
R[1] C[1]  
...  
R[NM-1] C[NM-1]
```

Aquí, $R[k]$ y $C[k]$ son el par de números enteros devueltos por la $-k$ -ésima llamada a `receive_block`.